Member-only story

AI + Knowledge Management + Open Source +

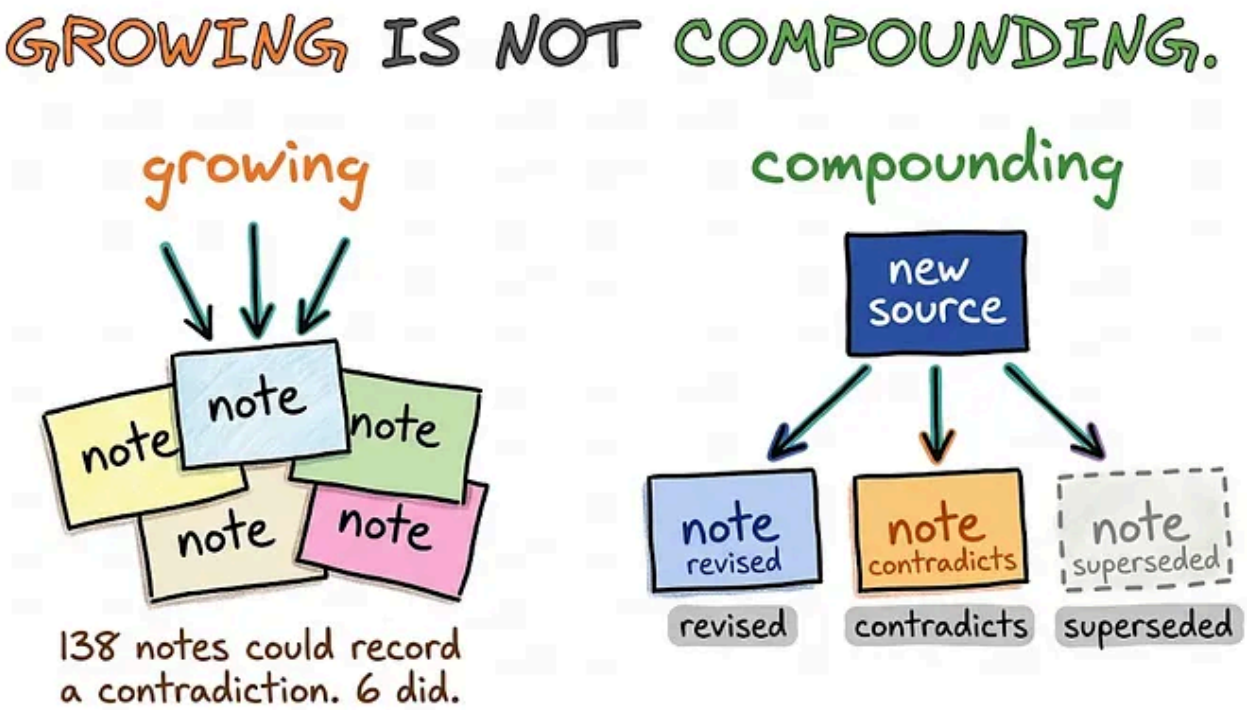
Six Rules that turn an OKF/AI-XF Folder Into a Compounding Intelligent Wiki



David R Oliver Follow 12 min read · Sep 22, 2026

234 2 1

Karpathy's LLM wiki says to compile knowledge once. OKF says label it. Neither says what to do when notes disagree. This rulebook does!



By the end of this piece, you will have six rules to paste into your agent's instructions, and four numbers to check. You will also have one command that computes those numbers, in an open format you can use today. Together they tell you whether your knowledge base compounds or just piles up. My own numbers come first, and they were worse than I expected.

My knowledge base has a place to record contradictions. It is a field called `contradicts`, and 138 of my notes carry it. Last week I counted how many are filled in.

Six.

All six sit in one small cluster of psychology notes. Not one of them concerns artificial intelligence (AI), the subject I read about every day. That field moves faster than any other I follow. Either it has never disagreed with itself, or my system hasn't been listening.

This essay is about that gap. Two of the best ideas in this young field both

Medium Search Play Games Write

The promise: knowledge that compounds

Start with the first idea. Andrej Karpathy is an AI researcher. He helped found OpenAI, the company behind ChatGPT, and he later led the self-driving software team at Tesla. When he shares a working habit, a great many engineers copy it.

In spring 2026 he shared a short note called LLM Wiki. An LLM, or large language model, is the kind of AI that powers chat assistants. His complaint was simple. Most tools search your documents afresh for every question, so they rediscover the same knowledge each time. Nothing builds up.

His fix is to have the model keep a wiki. You drop in a source. The model reads it, then updates a set of linked text pages. In his words: “The knowledge is compiled once and then kept current.”

He names three jobs. **Ingest** means read a new source and update the pages. **Query** means answer from the pages, and file good answers back. **Lint** is a programmer’s word for an automated tidy-up check. His lint pass hunts for contradictions and for “stale claims that newer sources have superseded”.

It is a fine pattern, and I have run a version of it for two years. But read his closing line again: “Your LLM can figure out the rest.”

The rest, it turns out, is the hard part.

The labels: what Google wrote down

Now the second idea. In June 2026, Google Cloud published the Open Knowledge Format, or OKF. It is a plain convention. Keep your knowledge as a folder of text files. Put a small block of labels at the top of each file. That block is called frontmatter.

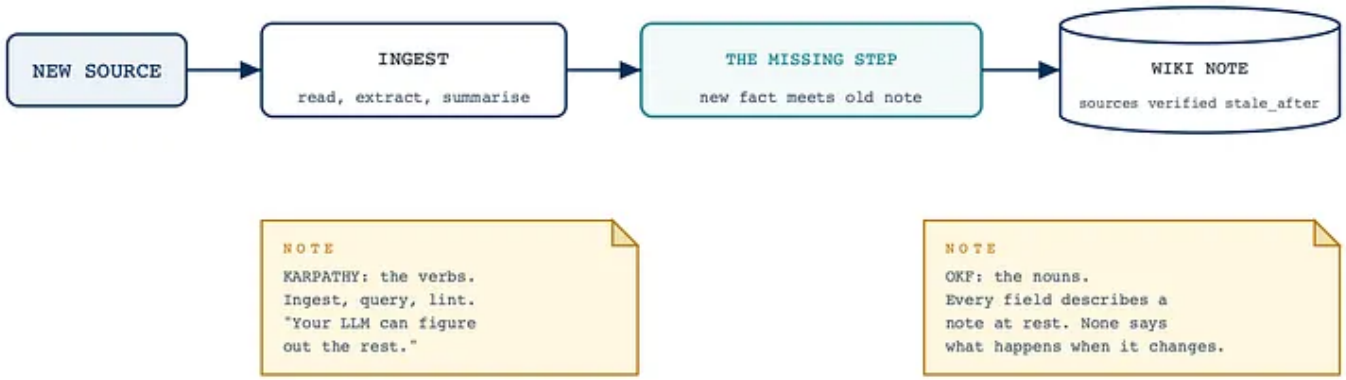
Six weeks later, version 0.2 added labels for trust. `sources` records where a note came from. `generated` records who wrote it, person or machine. `verified` records who checked it. `status` can be draft, stable or deprecated. `stale_after` is an expiry date for the content.

These are good labels, and I have written about them before. I even built a compatible format on top of them. It is called AI-XF, short for AI eXchange, and it adds typed links and permanent names. Hold that thought, because my format turns out to share the blind spot. But first, look at what kind of words these labels are.

They are all nouns. Each one describes the state of a note at rest. None describes what should happen when a note changes. You can mark a note deprecated, but no field says what replaced it. You cannot record that two notes disagree. Links between notes carry no type. The spec itself calls them an “untyped relationship”.

So here is the gap in one line. Karpathy gave us the verbs without the rules. OKF gave us the nouns without the verbs. Neither tells the model what to do when a new fact meets an old note.

That moment is where compounding happens, or fails to.



Karpathy names the verbs, OKF names the nouns, and the step where a new fact meets an old note belongs to neither.

What I found when I looked

I assumed my own vault was compounding. It holds about 620 active knowledge notes, fed by an ingestion pipeline I built. So I audited it. Three findings stung.

The contradiction field was decoration. You have seen the number: six of 138. The companion field, `supersedes`, records that one note replaces another. It was filled in zero times. My agent was adding knowledge. It was not arguing with what was already there.

The trust label had stopped meaning anything. 413 notes carry a confidence label. Of those, 303 say “high”. Just 10 say “low”. When nearly three notes in four claim top marks, the label carries no information. A grade that everyone gets is not a grade.

My history could not answer the key question. I wanted one ratio: how often does new reading update an old note, and how often does it create a new one? Git, the version-control tool that records every change to my files, should know that. It did not. Since June, nine new concept notes appeared. Meanwhile 162 existing ones changed, all inside a single bulk sync commit. Real revisions and routine file shuffling were mixed together, and I could not separate them.

Piling up feels like progress. The folder gets bigger. The graph gets denser. But interest only compounds when it is added back to the principal. Knowledge only compounds when new reading changes old notes.

The missing rulebook

Here is the rulebook my own ingestion agent now runs under. Think of it as the editorial policy that both sources leave to you. There are six rules.

1. Search before you write, and search by meaning

Before the model creates a page, it must look for an existing one. Most people do this with keyword search. That is not enough. The duplicate that hurts you is the one that uses different words. One note says “agent memory”. Another says “persistent context”. They cover the same idea, and keyword search will never introduce them.

So give every note a list of alternative names. Then run a second search that matches meaning, not spelling. Only after both may the model choose one of five moves: update, create, merge, split or deprecate. Make it write down which move it chose and why.

2. Never rewrite the evidence

Each time a model rewrites a page, it paraphrases. Each paraphrase loses a little. After ten rounds you have a photocopy of a photocopy, and it reads beautifully. A false detail that slips in on round three is a “known fact” by round four. Errors compound at the same rate as knowledge.

Karpathy keeps raw sources untouched, and that is right. Go one step further. Every claim on a wiki page should point at the source it came from. Then any page can be rebuilt from the evidence, and any sentence can be

challenged. OKF already has the mechanism. Give each source a short `id`, then cite it with a footnote beside the claim. The format allows this. Only your rules can insist on it.

3. Store claims, not topics

A note called “Voice Latency” can only grow longer. A note called “Voice Replies Under 200 Milliseconds Need Local Processing” can be proved wrong. That makes it far more useful.

A topic is a bucket. A claim is a position. New evidence can confirm a claim, weaken it or overturn it, and each of those is a real update. Keep topic pages as maps that point to claims. Let the claims do the compounding.

4. Treat a contradiction as a ticket, not an edit

When new reading disagrees with an old note, the model’s instinct is to smooth it over. It rewrites the paragraph so that everything agrees again. That instinct destroys the most valuable signal your system will ever produce.

So forbid it. The model must keep both versions and record a `contradicts` link between them, marked open. Then it stops. You are the judge. When you

rule, the link is marked resolved, with your name and the date. If one side won, the winner gets a `supersedes` link to the loser, and the loser is marked deprecated. My six-in-138 figure is what happens when this rule is missing.

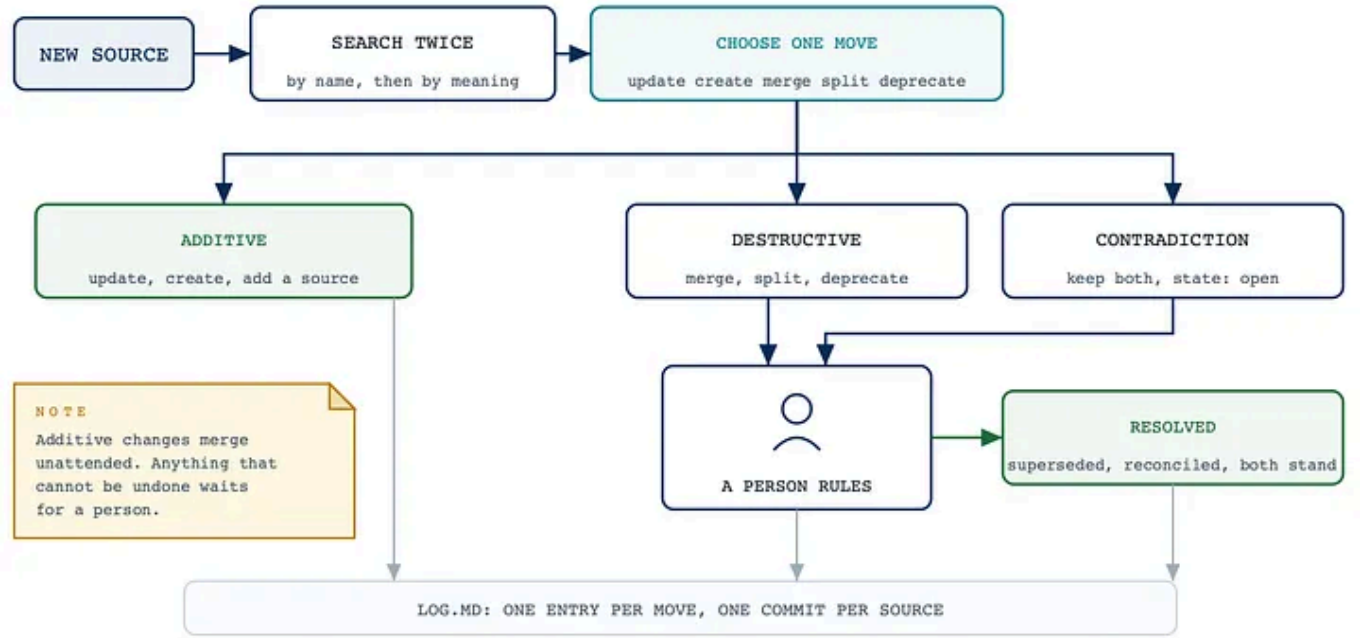
5. Gate by reversibility, not by size

Many teams let small changes through and review big ones. That is the wrong line. Adding a source looks small, but it can raise trust in a false claim.

Draw the line at what can be undone. Additions may merge on their own. A merge, a deletion or a deprecation waits for a person. So does any rise in a trust label. If you use OKF's `verified` stamp, let your review be the act that writes it. Then the stamp means a human looked.

6. One ingest, one commit

This dull rule makes the others measurable. Each source the model processes gets its own entry in version history. Name the source and list the moves made. My bulk sync commits are why I could not answer my own question. Keep file shuffling and knowledge changes in separate commits, always.



The six rules as one flow. Additive changes merge on their own. Anything destructive, and every contradiction, waits for a person. Every path lands in the log.

Four numbers that tell you the truth

Rules without measures drift. These four numbers take minutes to compute. The next section gives you one command that prints them.

Number	What it tells you	Warning sign
Updates per new note	Whether reading changes what you hold	Almost every ingest creates, few update
Contradictions opened and closed	Whether the system argues with itself	Zero opened, or a queue that never shrinks
Share of notes past `stale_after`	Whether old knowledge gets revisited	The share climbs every month
Questions answered from the wiki alone	Whether the wiki is earning its keep	You keep going back to raw sources

There is a fifth check, and it is free. Look at the spread of your trust labels. If nearly all say “high”, as mine do, reset them. A label has to be able to say no.

The fourth number deserves a note. Every question your wiki cannot answer is a gift. Log it. That list of failures is your reading list for next week. Karpathy files good answers back into the wiki. File the bad ones too, as gaps.

The shipped part: AIX v0.3

Pointing at gaps is cheap, so here is what I built this week. I started by holding my own format up to the six rules. It failed in an instructive way.

AIX already had a `contradicts` link. Its specification said the resolution "SHOULD be in prose". So a folder could record a disagreement, but could never answer a simple question: which disagreements are still open? It had no word for a merge, and none for evidence. Its change log was free text, so

you couldn't count anything in it. My format described notes at rest, exactly as Google's does.

AIX v0.3 describes notes in motion. Each change maps to a rule you have just read.

Contradictions get a life cycle. A `contradicts` link now carries a state, open or resolved. It records who raised it and when. A resolved one records who ruled, and how: one side won, both were edited, or both stand. A link with no state counts as open. If only a machine has ruled, a reader may still treat it as open. That is rule four, written into the file.

Merged notes leave a tombstone. When two notes turn out to be one, the absorbed note stays behind as a one-line stub. It is marked deprecated and carries a `merged-into` link to the survivor. Its permanent name is never reused. Old links keep working, and a reading agent is told to follow the trail to the successor.

Claims get evidence. There is a new `supports` link, the partner of `contradicts`. There is also a recommended note type called `Claim`. Together they let an agent see how a claim stands: what backs it, and what disputes it. That is rule three.

The change log becomes countable. Each entry in the log file now starts with one of nine fixed words. They include Creation, Update, Merge, Contradiction and Gap. The folder can now report its own ratio of updates to new notes. No version history is needed. That is the question my Git history could not answer.

One command prints the numbers. The checking tool gains a `--stats` option:

```
python3 tools/aix-validate.py path/to/your/folder --stats
```

It reports open and resolved contradictions, and the share of notes past their `stale_after` date. It reports how many sourced notes cite their evidence claim by claim. It reports the update ratio from the log. It also prints the spread of confidence labels, and flags a lopsided one. Mine would have been flagged at once.

The rulebook ships as a file. Karpathy describes a third layer in his pattern. He calls it the schema: the instructions that turn a chatbot into a disciplined wiki keeper. Both he and Google leave that layer to you. The repository now

holds one, in a file called `CURATOR.md`. It contains the six rules and four numbers from this essay, written to be pasted into your agent's instructions. It is not part of the standard. A format says what a folder can express, and a policy says how a keeper should behave. I have kept them apart on purpose.

And it is wired in. My vault has an ingestion pipeline. It reads each clipping, extracts its claims, checks them, and spawns notes. That pipeline now runs under the six rules. It searches by meaning and prefers updating an old note to creating a new one. It may append evidence to a note but never rewrite what is there. Novel claims become claim notes, titled as sentences that could be wrong. When a claim disagrees with an existing note, it keeps both, marks both, and puts the pair in a queue file for me to rule on. The pipeline may not merge, retire or delete a note, and it may not raise a trust label; it can only propose those. Each run writes one log line per move, in the same nine-word vocabulary as the format. The report at the end now prints the four numbers.

None of this cost new software. It was a page of instructions to the agent, one queue file, and one log file. That is the whole argument of the essay in miniature: the format was never the missing piece. The rulebook was.

The audit also embarrassed me in a smaller way. AIX claimed to adopt Google's trust labels unchanged. Checking line by line, I found three places where my examples spelt them differently. I had written `active` where Google says `stable`, and `uri` where Google says `resource`. My agent names were wrong too. Version 0.3 corrects all three, and the checking tool now warns on the old spellings. An unchecked claim of compatibility is just another unverified note.

Every folder that was valid under version 0.2 is still valid under 0.3. Every AIX folder is still a valid OKF folder. Nothing in the new version is compulsory.

What I am not claiming

The wiring is days old. I have the audit that exposed the problem and a pipeline that now follows the rules. I do not yet have the follow-up numbers that prove the cure. The queue starts empty, and the 620 existing notes are not being re-processed. I will publish the numbers when they exist, good or bad.

My vault has one author and one reader. Rule four assumes a judge with time to judge. In a team, that queue needs an owner, and I have not solved that.

The rules also cost money. Two searches and a written decision for each source is slower than dumping files in a folder.

This isn't a complaint about Karpathy or Google. Karpathy said plainly that his note was intentionally abstract. OKF is a format, and formats should stay small … I'm just pushing it all forward!

AIX still has one producer and one consumer, and both are me! That makes version 0.3 a published hypothesis, not a standard. My audit also proves something awkward about formats in general. Having the field is not the same as using it. My vault had a `contradicts` field for months, and the field changed nothing. Version 0.3 makes the gap visible and countable. It cannot close it. Only the discipline does that. Feedback is very welcome! Tell me what you would like to see in future versions of AIX or, for that matter, OKF!

The short version

A knowledge base compounds only when new reading is allowed to change old notes. That takes an editor with rules, and a model will not invent those rules for you. Write them down. Make contradictions visible. Keep the evidence apart from the summary. Then count.

Start with one number tonight. How many of your notes have ever been marked as contradicted?

Mine was six. Ask me again in three months.

I write about agentic AI, architecture and knowledge systems. Earlier pieces in this series covered why I built a superset of Google's format and why the folder has a ceiling. AI-XF v0.3, with the curator policy and the stats command, is on GitHub under MIT.

AI + Knowledge Management + Open Source +



Written by David R Oliver

2.3K followers · 322 following

I'm interest in how to use technology to improve peoples lives and the environment around us for the benefit of us all.

Follow

Responses (2)

Krzysztof

What are your thoughts?

Britt Whitaker
Sep 24

David - THANKS for this! I already had OKF integrated with my dev framework and AIX installed today already

has made my efficiency jump by at lease 3X! Very well done!

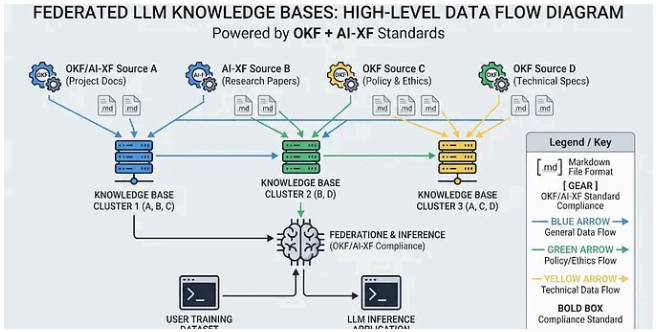
3 1 reply Reply

Mark W Kiehl
Sep 29

Thank you for your continued work on this. I use it daily.

Reply

More from David R Oliver





David R Oliver · Sep 27

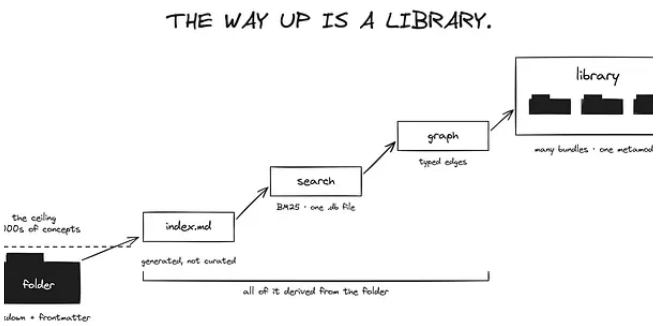
AI-XF v0.4 Adds Federation to Markdown Knowledge Bases

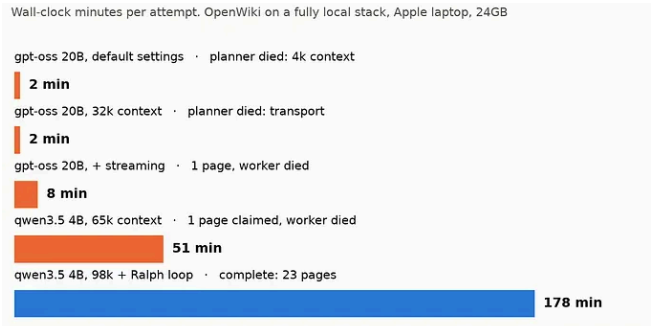
36 1

David R Oliver · Aug 2

Why Developers Use Macs, and Why the Data Says Something...

148 18 1





David R Oliver · Aug 1

OKF v0.2 Quietly Admits the Folder Has a Ceiling. The Way Up Is a...

402 6 1

David R Oliver · Aug 26

OpenWiki Turns Your Codebase Into Self-Correcting Memory

158 4 1

See all from David R Oliver

Help Status About Careers Press Blog Store Privacy Rules Terms Text to speech